

TestPlatform

Rapport d'Execution de Campagne

test

Projet: buggy shop

Version: 2.5

Environnement: Buggy_test

Debut: 17/06/2026 11:39 | Fin: 17/06/2026 11:41

Duree: 117.4 s

Rapport genere le 17/06/2026 11:41

ECHEC

Resume executif

Total	Succes	Echecs	Erreurs	Duree
1	0	1	0	117.4 s

Taux de reussite



Evolution vs campagne precedente: N/A

Premiere campagne

Corrections IA: 1 test(s) corrige(s) automatiquement, 0 avec succes

Informations du contexte

Projet	bugy shop
Description	test

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	N/A	main

Cas de test

Titre	Type	Priorite	Risque	Script
add Order	UNIT	2	CRITICAL	IA

Environnement

URL Web	http://localhost:8090
URL API	http://localhost:8090/api
Repository	https://github.com/ilyeschaabani/buggy-shop-api
Branche	main
Base de donnees	H2

Resultats details

Synthese par suite

Suite	Total	Succes	Echecs	Taux
Suite UNIT	1	0	1	0%

Suite UNIT

add Order		IA x3	FAILURE
Type	UNIT	Duree	45.9 s
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.456 s <<< FAILURE! -- in suites.unit.OrderServiceTest

Analyse IA

add Order

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.456 s <<< FAILURE! -- in suites.unit.OrderServiceTest [ERROR] suites.unit.OrderServiceTest.test_createOrder_happy_path -- Time elapsed: 0.035 s <<< FAILURE! java.lang.AssertionError: Subtotal should be price * quantity expected [1000.0] but found [502.0] [ERROR] Failures: [ERROR] OrderServiceTest.test_createOrder_happy_path:87 Subtotal should be price * quantity expected [1000.0] but found [502.0] [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project buggy-shop-api: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec-14662047120184451418\target\surefire- reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	**Cause** : Le test compile et s'exécute, mais échoue à la ligne 87 (<code>`assertEquals(response.getItems().get(0).getSubtotal(), ...)`</code>) car la méthode de calcul du sous-total dans <code>`OrderService.createOrder()`</code> effectue une addition (<code>`price + quantity`</code>) au lieu d'une multiplication (<code>`price * quantity`</code>). **Correction** : Corriger la logique de calcul du sous-total dans la méthode <code>`createOrder()`</code> de <code>`OrderService`</code> pour multiplier le prix (<code>`product.getPrice()`</code>) par la quantité (<code>`itemReq.getQuantity()`</code>), et vérifier que le total est bien la somme des sous-totaux, et que le stock est décrémenté (<code>stockInitial - quantity</code>). **Conseil** : Écrire des tests unitaires <i>*avant*</i> (TDD) ou immédiatement après l'implémentation des méthodes critiques permet d'identifier rapidement les erreurs de logique métier comme celles-ci.

Corrections automatiques par IA

L'IA a detecte des erreurs dans certains scripts generes et les a corriges automatiquement.

add Order

3 correction(s)

FAILURE

=== Tentative 1/3 ===

```
Erreur détectée: [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.483 s <<<
FAILURE! -- in suites.unit.OrderServiceTest
[ERROR] suites.unit.OrderServiceTest.test_createOrder_happy_path -- Time elapsed: 0.045 s <<< FAILURE!
java.lang.AssertionError: Subtotal should be price * quantity expected [1000.0] but found [502.0]
[ERROR] Failures:
[ERROR]   OrderServiceTest.test_createOrder_happy_path:86 Subtotal should be price * quantity expected
[1000.0] but found [502.0]
[ERROR] Tests run: 1,
Résultat: Script corrigé (2777 chars), relance Maven...
Résultat: Encore en échec (exit=1).
```

=== Tentative 2/3 ===

```
Erreur détectée: [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.444 s <<<
FAILURE! -- in suites.unit.OrderServiceTest
[ERROR] suites.unit.OrderServiceTest.test_createOrder_happy_path -- Time elapsed: 0.036 s <<< FAILURE!
java.lang.AssertionError: Subtotal should be price * quantity expected [1000.0] but found [502.0]
[ERROR] Failures:
[ERROR]   OrderServiceTest.test_createOrder_happy_path:81 Subtotal should be price * quantity expected
[1000.0] but found [502.0]
[ERROR] Tests run: 1,
Résultat: Script corrigé (3000 chars), relance Maven...
Résultat: Encore en échec (exit=1).
```

=== Tentative 3/3 ===

```
Erreur détectée: [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.457 s <<<
FAILURE! -- in suites.unit.OrderServiceTest
[ERROR] suites.unit.OrderServiceTest.test_createOrder_happy_path -- Time elapsed: 0.042 s <<< FAILURE!
java.lang.AssertionError: Subtotal should be price * quantity expected [1000.0] but found [502.0]
[ERROR] Failures:
[ERROR]   OrderServiceTest.test_createOrder_happy_path:87 Subtotal should be price * quantity expected
[1000.0] but found [502.0]
[ERROR] Tests run: 1,
Résultat: Script corrigé (3019 chars), relance Maven...
Résultat: Encore en échec (exit=1).
```

Metriques

Top 5 des tests les plus lents

Test	Suite	Duree
add Order	Suite UNIT	45.9 s

Temps d'execution par type

Type	Duree moyenne
UNIT	45.9 s

Tests instables

Aucun test instable detecte

Recommandations

- add Order:

****Cause**** : Le test compile et s'exécute, mais échoue à la ligne 87

(`assertEquals(response.getItems().get(0).getSubtotal(), ...)`) car la méthode de calcul du sous-total dans `OrderService.createOrder()` effectue une addition (`price + quantity`) au lieu d'une multiplication (`price * quantity`).

****Correction**** : Corriger la logique de calcul du sous-total dans la méthode `createOrder()` de `OrderService` pour multiplier le prix (`product.getPrice()`) par la quantité (`itemReq.getQuantity()`), et vérifier que le total est bien la somme des sous-totaux, et que le stock est décrémenté (`stockInitial - quantity`).

****Conseil**** : Écrire des tests unitaires **avant** (TDD) ou immédiatement après l'implémentation des méthodes critiques permet d'identifier rapidement les erreurs de logique métier comme celles-ci.

Annexe - Scripts generes par IA

add Order

```
package suites.unit;

import org.mockito.InjectMocks;
import com.demo.buggyshop.dto.OrderRequest;
import com.demo.buggyshop.dto.OrderResponse;
import com.demo.buggyshop.entity.Order;
import com.demo.buggyshop.entity.OrderItem;
import com.demo.buggyshop.entity.Product;
import com.demo.buggyshop.repository.OrderRepository;
import com.demo.buggyshop.repository.ProductRepository;
import com.demo.buggyshop.service.OrderService;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

import java.util.Arrays;
import java.util.Optional;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;
import static org.testng.Assert.*;

public class OrderServiceTest {

    @Mock
    private OrderRepository orderRepository;

    @Mock
    private ProductRepository productRepository;

    @InjectMocks
    private OrderService orderService;

    private AutoCloseable mocks;

    @BeforeMethod
    public void setUp() {
        mocks = MockitoAnnotations.openMocks(this);
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mocks.close();
    }

    @Test
    public void test_createOrder_happy_path() {
        // Given
        Long productId = 100L;
        Product product = new Product();
        product.setId(productId);
        product.setName("Laptop");
        product.setPrice(500.0);
        product.setStockQuantity(10);

        OrderRequest.OrderItemRequest itemReq = new OrderRequest.OrderItemRequest();
        itemReq.setProductId(productId);
        itemReq.setQuantity(2);

        OrderRequest request = new OrderRequest();
        request.setUserId(42L);
```

```
request.setItems(Arrays.asList(itemReq));
request.setShippingAddress("12 Rue de Paris");

when(productRepository.findById(productId)).thenReturn(Optional.of(product));
when(productRepository.save(any(Product.class))).thenAnswer(inv -> inv.getArgument(0));
when(orderRepository.save(any(Order.class))).thenAnswer(inv -> {
    Order o = inv.getArgument(0);
    o.setId(1L);
    return o;
});

// When
OrderResponse response = orderService.createOrder(request);

// Then
assertNotNull(response);
assertEquals(response.getUserId(), Long.valueOf(42L));
assertEquals(response.getShippingAddress(), "12 Rue de Paris");
assertEquals(response.getStatus(), Order.OrderStatus.PENDING);

// BUG 1: subtotal should be price * quantity = 500 * 2 = 1000
// but code does price + quantity = 500 + 2 = 502
assertEquals(response.getItems().get(0).getSubtotal(), Double.valueOf(1000.0),
    "Subtotal should be price * quantity");

// BUG 2: totalAmount should be 1000.0
assertEquals(response.getTotalAmount(), Double.valueOf(1000.0),
    "Total should equal sum of subtotals");

// BUG 3: stock should be decremented (10 - 2 = 8)
// but code does 10 + 2 = 12
verify(productRepository).save(argThat(p ->
    p.getStockQuantity() == 8
));
}
```